Redis

Redis 自稱為「資料結構儲存」，用 C 語言實作。在記憶體中運行（in-memory），並採用單執行緒（single-threaded）模式，因此讀寫速度非常快，且行為相對容易推理。對於希望追求效能與簡單性的系統設計，這樣的特性是非常合適的。

值得注意的是，Redis 的設計重點在於速度，因此它的耐久性（durability）不像關聯式資料庫一樣強。如果你需要強一致性的持久化策略，可以透過 Redis 的 AOF（Append-Only File）機制來減少資料遺失，但這仍然不等同於傳統資料庫的確保提交寫入磁碟的保證。這是一個明確的設計取捨：Redis 優先考量速度。若需要在速度與耐久性間取得平衡，你也可以選擇像 AWS MemoryDB 這類在速度之餘也支持磁碟耐久性的實作。

Redis 支援的基礎資料結構

Redis 底層仍是以鍵值（key-value）儲存為核心，但值（value）可以是各種資料結構：

- String（字串）
- Hash（哈希 / 物件結構）
- List（列表）
- Set（集合）
- Sorted Set（排序集合，用於優先隊列）
- Bloom Filter（布隆過濾器，用於集合成員性測試）
- Geospatial Indexes（地理空間索引）
- Time Series（時間序列）

這些資料結構本質上對應著程式設計語言中常見的資料類型，讓你在使用時可以直接把資料當成這些結構操作，而不是簡單的 binary blob。

除了基本資料結構，Redis 也支援一些進階的通訊模式，例如 Pub/Sub（發布 / 訂閱）與 Streams（串流），可以用來模擬或部分取代像 Kafka 或 AWS SNS/SQS 這種消息隊列系統。

Redis 的指令（Commands）

buildmoat.org

Redis 使用自定義的 wire protocol，所有功能都是用簡單的字串命令來實現。你可以用 CLI 直接連線 Redis 實例並下指令：

```
SET foo1
GET foo# 回傳 1
INCR foo# 回傳 2
XADD mystream * name Sara surname OConnor# 向串流新增資料
```

這些命令在不同的資料結構之下有不同的語意。例如針對 Set 的操作包括：

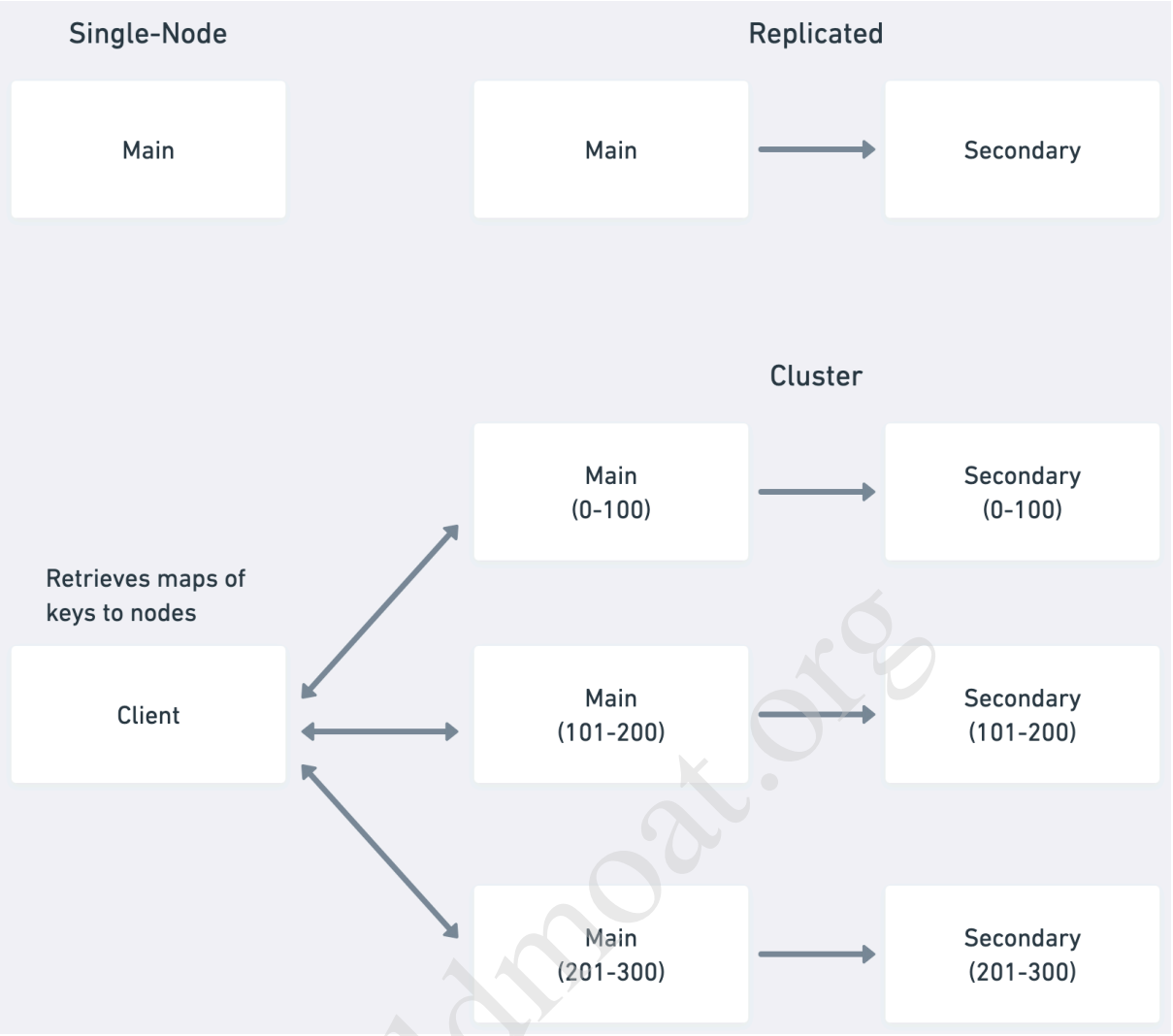
```
SADD myset 值1
SCARD myset# 取得集合大小
SMEMBERS myset# 列出所有成員
SISMEMBER myset 值1# 是否為成員
```

Redis 的命令集合相對可讀，使得理解其行為變得容易。

Redis 的基礎架構部署模式

Redis 可以用在下列幾種架構模式：

buildmoat.org



單節點模式（Single Node）

最基本的部署方式，一個 Redis 實例處理所有資料。

高可用模式（HA Replica）

可以配置主節點與一個或多個副本節點，以提高讀取能力及容錯性。

Cluster 模式

當 Redis 運行於 Cluster 時，會將所有 key 分成多個 hash slot，Client 端會維持一份這些 hash slot 到 node 的對照表，藉此直接連線到含該 key 的節點。Cluster 的設計讓 Redis 可以水平擴展，但需要注意的是 Redis cluster 預設只支援單鍵的操作，跨鍵的複雜操作需確保 key 屬於同一節點。

如果 client 請求錯誤的節點，該節點會回傳 MOVED 指令，Client 會重新更新本地的 hash slot 映射再轉向正確節點。Cluster 節點之間也會透過某種程度的 gossip 協定進

buildmoat.org

行節點狀態的傳播，但 Redis 的設計重點仍然是性能，因此 Cluster 的複雜度不像傳統分散式資料庫那麼高。

Redis 的效能

Redis 的效能非常高，單一實例可以容納每秒 O(100,000) 級別的寫入請求，且讀取延遲通常落在微秒級。這些特性使得在某些其他資料庫看來是反模式的操作，在 Redis 中會變成可以接受或者可行的，例如為了讀取一個清單而發出 100 次查詢操作，這在 SQL 系統中肯定會很糟糕，但在 Redis 中成本相對較低。

Redis 常見使用情境（Capabilities）

Redis 作為 Cache

Redis 作為快取是最常見的部署情境。Redis 每個 key/value 對應一筆快取資料，你可以透過水平擴展 Cluster 來加強容量。當需要更多容量時，只要增加節點即可。

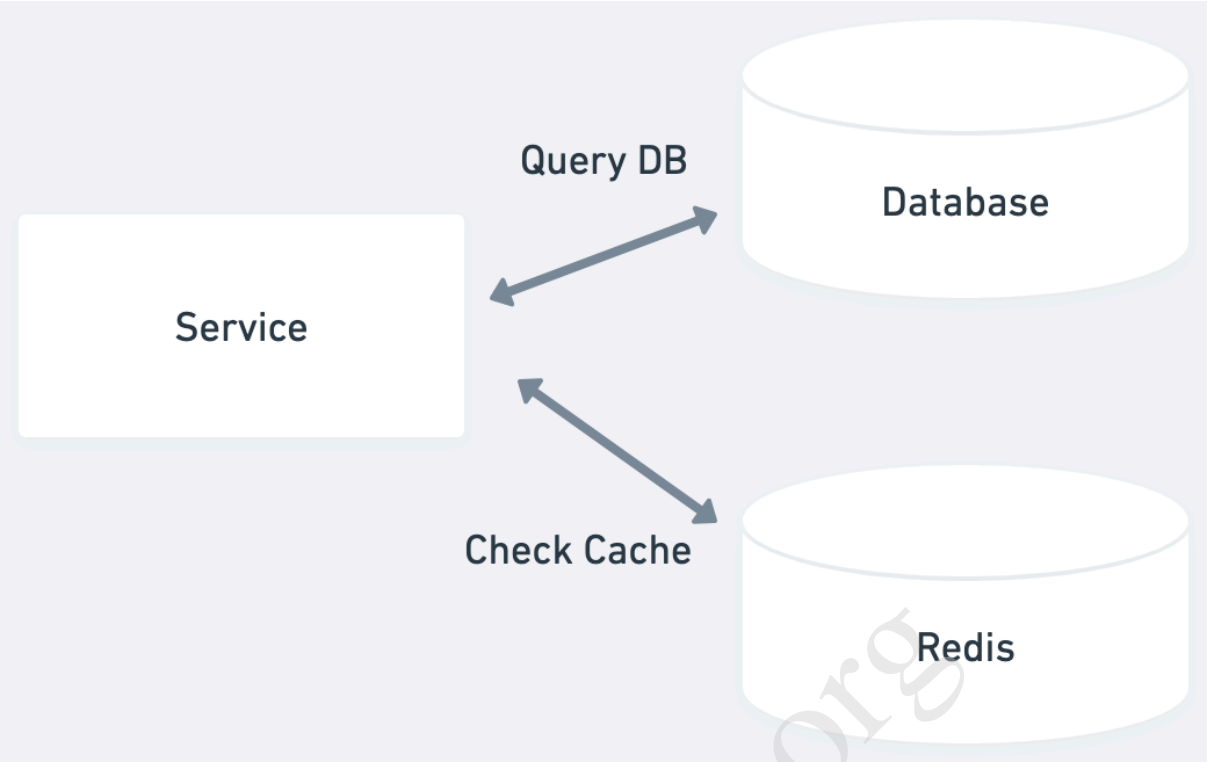
例如你可能 Cache 某商品：

```
key: product:123value: JSON 物件或 Redis Hash
```

為了管理快取大小與過期時間，Redis 提供 TTL（Time To Live），當 TTL 到期以後該 key 就會被移除。使用 TTL 可以避免 cache 過大。

即使 Redis 本身不會自動解決所有緩存問題，例如「熱 key（hot key）」問題，但這也是分散式快取常見的情況，需要設計合適的策略來應對。

buildmoat.org



Redis 作為 Distributed Lock（分布式鎖）

在系統設計中另一個常見用途是使用 Redis 實作分布式鎖。在某些需要維持資料一致性的更新流程，或確保同一個操作不會被多個實例同時執行的場景（例如 Ticketmaster、Uber 類問題），分布式鎖可以派上用場。

Redis 中可以用原子性的 INCR 加上一個 TTL 來實作一個非常簡單的鎖機制：當 INCR 回傳值為 1 時即代表第 1 個取得鎖，其他嘗試者必須等待或重試。完成後可以刪除該 key 來釋放鎖。更進階的實作可以使用 Redlock 演算法並搭配 fencing token 等技術來避免鎖失效與造成競態條件。

Redis 作為 Leaderboard（排行榜）

Redis 的 Sorted Set 支援以 log(N) 級時間複雜度維護排序資料，這使得它很適合用於排行榜應用。利用高寫入吞吐量與低讀取延遲，可以即時維護例如：

```
ZADD tiger_posts500"SomeId1"
ZADD tiger_posts1"SomeId2"
ZREMRANGEBYRANK tiger_posts0 -6# 只保留前 5 名
```

這在對即時排行榜或類似功能的設計中非常常見。

Redis 的 Rate Limiting（流量限制）

buildercat.org

Redis 也可以用來實作不同的 rate limiting 演算法，例如定量視窗（fixed window）。基本做法是：

```
INCR key# 增加計數
如果 > N，則限制請求
EXPIRE key W# W 秒後 reset
```

滑動視窗（sliding window）可以用 Sorted Set 來存放 timestamp，再搭配 Lua script 保證原子性操作。

Redis 的 Proximity Search（鄰近搜尋）

Redis 原生支援地理空間索引與查詢：

```
GEOADDkey longitude latitude member
GEOSEARCHkey FROMLONLAT longitude latitude BYRADIUS radius
unit
```

Redis 使用 geohash 作為底層索引方式，雖然在某些情況下搜尋結果需要進一步過濾，但它仍然是一個可行的解決方案。

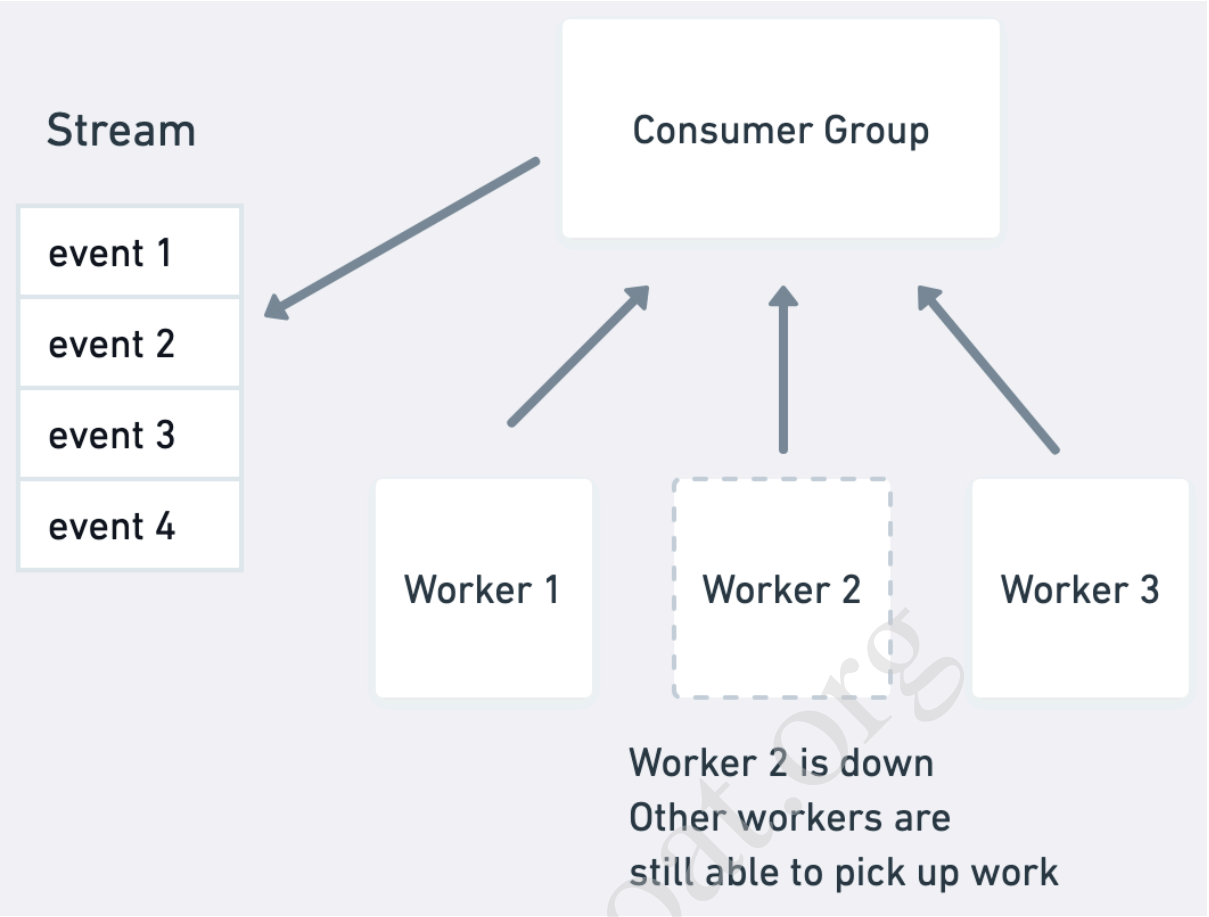
Redis 作為 Event Sourcing（事件來源）

Redis 的 Streams 提供類似 Kafka 的 append-only 日誌結構，可以搭配 consumer group 來實作可靠的工作隊列模式。其中：

```
XADD stream *field value
XREADGROUP GROUP groupName consumer ...
XCLAIM# 確認失敗 consumer 的工作
```

Redis Streams 允許在某個 consumer 失敗後，讓其他 consumer 來接手處理未完成的項目。

buildnoat.org



buildmoat.org

Redis 的 Pub/Sub（發布/訂閱）

Redis 的 Pub/Sub 讓客戶端可以訂閱某頻道來接收即時訊息。指令包括：

```
PUBLISH channel message
SUBSCRIBE channel
```

這對於聊天室、通知系統等即時通訊場景很有用。但請注意，Pub/Sub 是 *at most once* 的消息交付——如果 subscriber 在訊息發布時離線，就會漏掉訊息。若需要更可靠的消息持久化或重播功能，應該考慮 Redis Streams 或其他消息系統（Kafka、SNS/SQS）。

當客戶端訂閱某個 channel 之後，只要連線仍然保持開啟，它就會收到所有發佈到該 channel 的訊息。這種模型非常適合「短暫性（ephemeral）」、即時性的通訊場景，例如聊天室、即時通知、線上狀態更新等等。

但有一個非常重要的特性要理解：**Pub/Sub 不會持久化訊息。**

如果某個訂閱者在訊息發佈當下是離線狀態，那麼它會完全錯過那條訊息。Redis 不會替你保存，也不會在重新連線時補發。這和 Kafka 或 Redis Streams 這類具備持久化與重播能力的系統完全不同。因此在系統設計時必須明確區分：

- 需要可靠投遞與重播 → 不適合純 Pub/Sub
- 只需要即時推播、允許偶爾遺失 → Pub/Sub 很合適

另一個實務上容易被忽略的細節是連線模型。

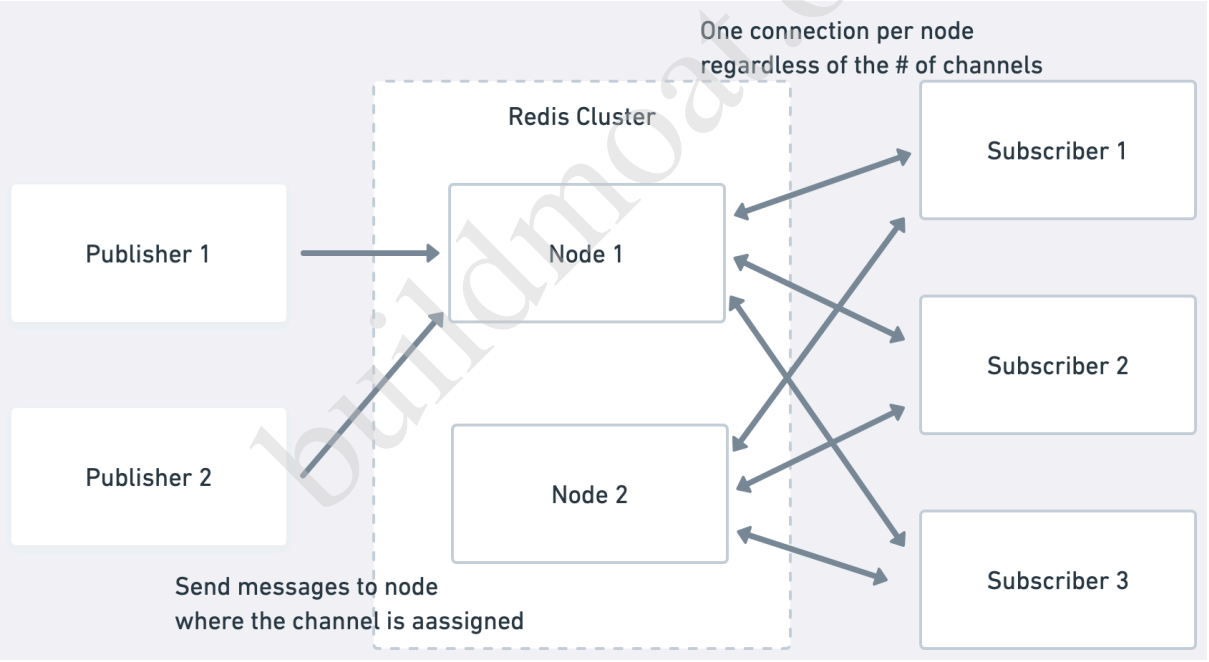
Pub/Sub 客戶端在 Cluster 模式下，**對每個節點只需要一條連線，而不是每個 channel 一條連線**。換句話說：

- 連線數量 ≈ cluster 節點數量
- 而不是 channel 數量

這點非常重要，因為它意味著：

即使你有數百萬個 channel，也不需要建立數百萬條 TCP 連線。只要維持與每個節點的一條連線即可。

這讓 Pub/Sub 在大規模 channel 數量場景下仍然可行，至少從連線資源消耗的角度來看是如此。



buildmoat.org

Redis 的缺點與改善策略

Redis 常見的一個問題是 **hot key**：當某一個 key 的請求量遠高於其他 key 時，可能導致該節點負載過高。解決方法包括：

- 在客戶端加快取層
- 將相同資料分散為多個 key（隨機化請求）
- 加上讀取副本並動態擴容

在面試設計中，提到 hot key 問題並提供對應策略會為設計加分。

小結

Redis 是一種強大、靈活、語意清晰的資料結構服務，它不只是快取，更可以在許多系統設計場景中擔任重要角色：

- 缓存
- 分布式鎖
- 排行榜
- 限流
- 地理搜尋
- 事件來源
- 即時通訊（Pub/Sub）

因為它的基礎資料結構對應程式語言中的常見型別，且架構相對簡單，能讓你在系統設計訪談中深入討論具體實作與可擴展性問題，而不用花太多時間理解複雜內部細節。

buildmoat.org